



Southwest WI Technical College

10-152-119 Intro to Algorithms

Course Design

Course Information

Alternate Title	Algorithmic Thinking for AI-Augmented Engineers
Description	Learners are introduced to algorithms as structured approaches to solving problems. The course focuses on analyzing problems, selecting appropriate algorithmic strategies, and implementing solutions using common data structures. Students explore how different approaches impact performance through basic analysis techniques and comparison of solutions. Emphasis is placed on reasoning, debugging, and refining solutions, both manually and with AI-assisted tools. By the end of the course, learners are able to evaluate and adapt algorithmic approaches to solve problems effectively.
Career Cluster	Information Technology
Instructional Level	A.A.S. - Associate in Applied Science
Total Credits	3
Total Hours	72

Types of Instruction

Instruction Type	Credits/Hours
(A) Lecture	36
(B) Lab	36

Pre/Corequisites

Textbooks

50 Algorithms Every Programmer Should Know: Tackle computer science challenges with classic to modern algorithms in machine learning, software design, data systems, and cryptography - 2nd ed. Edition - Imran Ahmad

Course Competencies**1. Analyze problems to define algorithmic solutions****Assessment Strategies**

- 1.1. Project assignments and quiz

Criteria

- 1.1. Decompose problems into clear, ordered steps
- 1.2. Distinguish between well-defined and ambiguous problem statements
- 1.3. Identify inputs, outputs, and constraints of a problem
- 1.4. Select appropriate algorithmic approaches based on problem characteristics
- 1.5. Represent solutions using structured logic (pseudocode, flow, or code)

Learning Objectives

- 1.a. Define an algorithm and its key characteristics
- 1.b. Explain problem decomposition and abstraction
- 1.c. Identify common problem-solving patterns
- 1.d. Describe inputs, outputs, and constraints in a computational problem

2. Implement algorithmic solutions using programming constructs and AI-assisted tools**Assessment Strategies**

- 2.1. Project assignments and quiz

Criteria

- 2.1. Implement algorithms using Python programming constructs
- 2.2. Utilize AI-assisted tools to generate or refine code solutions
- 2.3. Translate algorithmic logic into executable programs
- 2.4. Modify and adapt generated code to meet problem requirements
- 2.5. Maintain readable, structured, and functional code

Learning Objectives

- 2.a. Identify Python constructs used in algorithm implementation
- 2.b. Describe how AI-assisted tools can support development
- 2.c. Demonstrate translating logic into code
- 2.d. Explain code readability and structure principles

3. Evaluate algorithm correctness and efficiency using basic performance measures**Assessment Strategies**

- 3.1. Project assignments and quiz

Criteria

- 3.1. Verify correctness of algorithmic solutions through testing
- 3.2. Identify logical errors and inefficiencies in code
- 3.3. Compare multiple solutions to the same problem
- 3.4. Apply basic performance reasoning (e.g., relative efficiency, Big-O intuition)
- 3.5. Assess trade-offs between simplicity, performance, and readability

Learning Objectives

- 3.a. Define correctness in algorithmic solutions
- 3.b. Explain basic performance concepts (e.g., time complexity)
- 3.c. Identify common sources of inefficiency
- 3.d. Describe trade-offs in algorithm design

4. Compare and refine solutions using manual and AI-assisted approaches

Assessment Strategies

- 4.1. Project assignments and quiz

Criteria

- 4.1. Develop solutions independently before using AI assistance
- 4.2. Generate alternative solutions using AI-assisted tools
- 4.3. Compare manually developed and AI-generated solutions
- 4.4. Identify inaccuracies, limitations, or inefficiencies in AI-generated outputs
- 4.5. Refine solutions based on comparison and analysis

Learning Objectives

- 4.a. Describe differences between manual and AI-assisted solution approaches
- 4.b. Explain the role of AI as a development aid
- 4.c. Identify risks of relying solely on generated solutions
- 4.d. Demonstrate iterative improvement of solutions

5. Adapt solution strategies based on technical and organizational constraints

Assessment Strategies

- 5.1. Project assignments and quiz

Criteria

- 5.1. Implement solutions without reliance on AI-assisted tools when required
- 5.2. Adjust solution approaches based on tool availability or constraints
- 5.3. Communicate algorithmic solutions clearly to different audiences
- 5.4. Justify chosen approaches using technical reasoning
- 5.5. Translate solutions between different development contexts

Learning Objectives

- 5.a. Describe constraints that impact solution design
- 5.b. Explain differences between tool-assisted and manual development
- 5.c. Identify strategies for communicating technical solutions
- 5.d. Demonstrate adapting solutions to varying environments